

CSCE 614 COMPUTER ARCHITECTURE · MODULE 08 · LECTURE NOTES

Out-of-Order Execution

Running instructions early while pretending you did not.

MODULE OUTCOMES

- Explain register renaming and why it removes false dependences.
- Describe the reorder buffer and in-order retirement.
- Explain what limits instruction-level parallelism.
- Explain memory-level parallelism and why it matters more than ILP.
- Relate out-of-order behaviour to measured IPC.

1 Why execute out of order

An in-order pipeline stalls at the first instruction that cannot proceed, and everything behind it waits even if it is completely independent.

```
ld    r1, [r2]      ; cache miss: 250 cycles
add   r3, r1, r4     ; depends on r1 -- must wait (correct)
mul   r5, r6, r7     ; independent -- no reason to wait
sub   r8, r9, r10    ; independent
```

During a 250-cycle miss, a 4-wide core could have executed roughly a thousand instructions. An in-order machine executes none. The motivation for out-of-order execution is not elegance; it is that the alternative wastes an enormous fraction of the machine whenever memory is touched.

2 Register renaming

Module 03 distinguished true dependences (RAW — real information flow) from false ones (WAR and WAW — artefacts of two instructions happening to use the same register name). Renaming is the mechanism that acts on that distinction.

```

Before renaming -- r1 is reused:
ld  r1, [r2]      ; writes r1
add r3, r1, r4     ; reads r1  -- RAW, genuine
ld  r1, [r5]      ; writes r1  -- WAW with the first load
add r6, r1, r7     ; reads r1

After renaming to physical registers:
ld  p40, [p12]
add p41, p40, p13  ; still dependent -- RAW is real
ld  p42, [p14]     ; now INDEPENDENT of the first load
add p43, p42, p15

```

The processor maintains a far larger set of physical registers (150–400) than the ISA exposes (16 on x86-64), plus a mapping table. Every write to an architectural register allocates a fresh physical register; every read is redirected to whichever physical register currently holds that value.

WHAT THIS BUYS

False dependences disappear entirely, because register names are never reused. True dependences remain, because they represent actual data flow and no amount of renaming can remove them.

In the example, the two load chains become independent, so **both loads can be outstanding at the same time**. That is memory-level parallelism, and §4 argues it is the single most valuable thing the mechanism provides.

3 Keeping the illusion of sequential execution

Instructions **issue** and **execute** out of order, whenever their operands become available. They **retire** — commit their results to architectural state — strictly in program order, from the **reorder buffer**.

In-order retirement is what makes the whole scheme safe, and it provides two essential properties:

- **Precise exceptions.** If instruction 50 faults, instructions 1–49 have retired and 51 onward have not, regardless of what order they actually executed in. The exception appears exactly where the program says it should, which is what makes debugging and signal handling possible at all.
- **Speculation recovery.** Work done down a mispredicted branch is sitting in the reorder buffer unretired. Discarding it is simply a matter of flushing those entries — nothing architecturally visible ever happened. This is precisely the mechanism Module 04 relied on.

Resource	Typical capacity	Exhausted when
Reorder buffer	~500 entries	A long-latency miss sits at the head and blocks retirement while execution continues behind it, filling the buffer.
Physical registers	200–400	Many values are simultaneously live.
Load/store queue	70–130 entries	Many memory operations are in flight.

Resource	Typical capacity	Exhausted when
Scheduler (reservation stations)	100–200 entries	Many instructions are waiting on operands.

The **instruction window** is bounded by whichever of these fills first. A cache miss at the head of the reorder buffer is the classic stall: the core continues executing younger instructions, but none can retire, and when the buffer fills, everything halts. This is why a single long-latency miss on the critical path can cost far more than its nominal latency.

4 ILP, MLP, and which one matters

Instruction-level parallelism is how many independent instructions are available to execute simultaneously. Decades of research established that real code has limited ILP — dependence chains, branches, and memory ambiguity hold practical values to roughly 4–6, which is why core widths stopped growing aggressively.

MEMORY-LEVEL PARALLELISM IS THE ONE THAT PAYS

MLP is how many cache misses are outstanding simultaneously.

Serialised: ten misses, one at a time, 250 cycles each = 2,500 cycles.

Overlapped: ten misses issued together = roughly 250 cycles total, because the memory system handles them concurrently.

A factor of ten, from the same ten misses. No amount of arithmetic reordering comes close to that.

So the real purpose of out-of-order execution is to look far enough ahead to *find independent memory accesses and issue them at the same time*. The arithmetic reordering is almost incidental.

4.1 Consequences for code

- **Independent loads are valuable.** Traversing four arrays at once, or using several independent pointers, allows several misses to overlap. Processing them one array at a time does not.
- **Pointer chasing has MLP of exactly 1.** The address of the next node is not known until the current load returns, so no two misses can ever overlap. This is the deepest level of the array-versus-linked-list story: Module 02 explained the dependence, Module 05 the cache lines, and this module the lost parallelism.
- **A dependence chain running through a miss is the worst case.** The reorder buffer fills with instructions that cannot retire, and the window collapses.
- **Hyperthreading exists for this.** When one thread stalls on memory, a second thread supplies genuinely independent work to keep the execution units busy. It helps most on memory-bound code and can hurt on code that already saturates the core.

5 Reading IPC

Measured IPC	Interpretation	Where to look next
3–4	Near the machine's peak issue rate. Compute-bound.	Reduce the work itself, or vectorise it (Module 09). Memory optimisation will not help.
1–2	Moderate stalling.	Dependence chains, branch mispredictions, or partial memory stalls. Check branch-misses first since it is cheap to rule out.

Measured IPC	Interpretation	Where to look next
0.3–1	Substantial stalling.	Almost certainly memory. Check L1, L2, and LLC miss rates.
Below 0.3	Severely memory-bound.	Data layout and access pattern (Module 06). Also check TLB misses (Module 07).

IPC is the first number to obtain for any performance question because it immediately partitions the search space: either the core is working, in which case you must do less work, or it is waiting, in which case you must find out for what.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Onur Mutlu — Out-of-Order Execution, Tomasulo's Algorithm	Renaming, reservation stations, and the reorder buffer developed in full.
Agner Fog — The microarchitecture of Intel, AMD and VIA CPUs	Actual window sizes, physical register counts, and execution port layouts for every recent core.
Travis Downs — performance blog	Careful microbenchmark-driven investigations of out-of-order behaviour. An excellent model of how to measure this material.
Intel — Top-Down Microarchitecture Analysis Method	A systematic method for attributing stalls to front end, back end, bad speculation, or retiring. The professional version of §5.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Write a loop with a long chain of dependent additions and one with the same number of independent additions. Measure IPC for both and explain the difference.
- 2 Construct a memory-level parallelism experiment: traverse one pointer chain, then two, four, and eight independent chains simultaneously. Plot throughput against chain count and find where it saturates — that is your machine's MLP limit.
- 3 Demonstrate the reorder-buffer limit: place a cache miss followed by increasing numbers of independent instructions, and find the point at which adding more stops helping.
- 4 Measure the same memory-bound kernel with hyperthreading enabled and disabled. Then repeat with a compute-bound kernel. Explain both results.

- 5 Use perf stat to compute IPC for five programs of your own, and classify each using the table in §5. Verify your classification by checking the corresponding counter.
- 6 Write a linked-list traversal and an equivalent array traversal with identical element counts and working-set size. Measure both and attribute the difference to MLP rather than to cache behaviour alone — you will need to control for line utilisation.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What is out-of-order execution actually for?
- 2 What does register renaming remove, and what can it not remove?
- 3 Why must instructions retire in order when they execute out of order? Name the two properties this provides.
- 4 What is the instruction window, and what exhausts it first during a cache miss?
- 5 Define MLP and explain why it matters more than ILP.
- 6 Why does pointer chasing have MLP of 1, and what are the three levels of explanation for why linked lists are slow?
- 7 Your program shows IPC 0.2. What do you check next, and why?